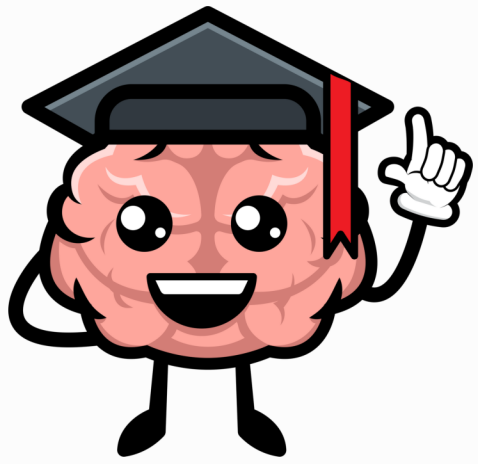
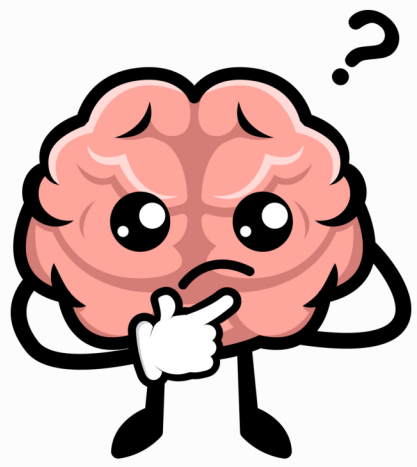
Smart  vs 

Dumb Components in Angular

Build clean, scalable apps with better component design

Niyati Kaneriya

Why this matters ?

Spaghetti Logic	Clean Architecture
Logic + UI mixed together	Clear separation of concerns
Hard-to-reuse components	Easier to maintain
Difficult to test	Scales better with teams
Slower debugging	Testable components

What is a Smart Component?

It does the thinking so others don’t have to.

- Handles logic
- Talks to services
- Owns data flow
- Often page-level (container)

```
ngOnInit() {
  this.posts$ = this.postService.getPosts();
}
```

Fetching data directly in the component and passing it to children

What is a Dumb Component?

It focuses on displaying, not thinking.

- Presentation only
- Receives input
- Emits output
- No logic inside

```
<post-card [post]="post" (like)="onLike($event)"></post-card>
```

Stateless component that gets data from a parent and emits events.

The Golden Rule

SMART = **DUMB**

Knows things

Shows things

Use Dumb components as much as possible, they are easy to reuse, simpler to test, and cleaner to maintain

Why Split Components?

Key Benefits of Separation

♻️ Reusability

- Dumb components are modular and easy to reuse across pages.

🧪 Testability

- Presentational components are easy to unit test — no services, no side effects.

📏 Clear Responsibilities

- Each component has one job — logic or display — not both.

🐞 Easier Debugging

- Problems are easier to isolate when logic and UI are separated.

Common Mistakes

❌ **Mixing Logic with UI**

- Business logic in UI components = messy code.

❌ **Using Services in Dumb Components**

- Keep Dumb components clean — no services, just @Input() / @Output().

❌ **Mutating @Input()**

- Never change input data directly. Treat it as read-only.

❌ **Skipping @Output()**

- Use @Output() to send events. Don't rely on hacks.

✅ **Keep it Clean:**

- *One component = one responsibility*.